

Demian RAG Pipeline v3

LangChain · Chroma · Hybrid Retrieval · RAGAS 평가

— 계측으로 다시 쓴 완성본 —

과제	GENAI_DAY4_RAG_프로젝트 — § 6 완성 예제 (빈 코드셀 10개)
제출	추병곤
원 강의자료	박광석 (모두의연구소)
구성	Step 1 Loader → Step 2 Splitter → Step 3 Embedding → Step 4 Retriever → Step 5 QA → 정량 평가(RAGAS)

```
Demian.pdf (스캔 OCR · 182p)
| PyPDFLoader.load() + clean()      분철 373건 복원 · 러닝헤더/워터마크 제거 [A]
▼ 유효 180p
페이지 병합 → RecursiveCharacterTextSplitter [B]
|           400 tok / overlap 80 / length=tiktoken_len
|           → 문자 오프셋 역매핑으로 page 복원
▼ 191 chunks
OpenAIEmbeddings (text-embedding-3-small, 1536d) [C]
▼
Chroma (persist, HNSW) [D]
▼
Hybrid Retriever = BM25(0.4) ⊕ MMR(0.6) k=4, fetch_k=20, λ=0.5 [1→3]
▼
Grounding Prompt (context-only · 근거 없으면 거절 · [p.# ] 인용) [4]
▼
gpt-4o (T=0) → 답변 + 출처 [5]
▼
RAGAS: precision / recall / faithfulness / relevancy
```

v3에서 바뀐 것 — 실측으로 확인한 두 가지를 반영했다. ① `chunk_size=500` 은 이 문서에서 한 번도 작동하지 않는 파라미터였다 (페이지 최대 ≈ 370 토큰 → chunk = page, overlap 미적용). ② 페이지의 78%가 문장 중간에서 끝난다. 그래서 페이지를 병합한 뒤 토큰 기준으로 다시 자르고, 인용 가능성은 오프셋 역매핑으로 유지했다.

last modified date : 2026.03.15

제작 : 박광석 (모두의연구소)

랭체인으로 RAG 시작하기

해당 노트는 Langchain으로 RAG를 구현하기 위해 필요한

각 컴포넌트인 Document Loaders, Text splitters, Text embeddings, Vectorstores, Retriever를 다룹니다

Step 0 : 설치와 준비

Langchain 설치 및 Gemini API 키를 등록하도록 합니다.

```
In
from google.colab import drive
drive.mount('/content/drive')
```

실행 결과

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive",
force_remount=True).
```

```
In
!pip install -U -q langchain langchain-openai
!pip install -U -q langchain-community langchain-core
!pip install -U langchain-text-splitters
```

실행 결과

(패키지 설치 로그 생략)

```
In
import os
```

```
In
from google.colab import userdata
os.environ['OPENAI_API_KEY'] = userdata.get('OPENAI_API_KEY')
```

```
In
#! curl ipinfo.io
```

```
In
from langchain_openai import ChatOpenAI

# OpenAI API를 사용하는 설정으로 변경
# 모델명은 필요에 따라 "gpt-4o", "gpt-4o-turbo", "gpt-3.5-turbo" 등으로 바꿀 수 있습니다.
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.0,
)
```

```
In
!pip install -q pypdf pdf2image docx2txt pdfminer unstructured #의존성 모듈을 설치합니다
```

Step 1 : Document Loaders 사용해보기

Document Loader는 다양한 형태의 원본 데이터를

LLM이 이해할 수 있는 Document 객체(text + metadata) 로 변환하는 역할을 합니다.

PDF, 웹페이지, CSV와 같이 형식이 서로 다른 문서들을 일관된 구조로 파싱하여, 이후 Chunking·Embedding·검색(Retrieval) 단계에서

바로 사용할 수 있도록 만들어줍니다.

즉, Document Loader는

RAG 파이프라인의 가장 첫 단계에서 “데이터를 읽을 수 있는 형태로 정리하는 역할을 담당합니다.

공식 문서에서는 지원되는 다양한 Loader 목록을 확인할 수 있습니다.

https://python.langchain.com/docs/modules/data_connection/document_loaders/

PDFLoader 사용

이번 실습에서는 가장 많이 사용되는 문서 형식인 PDF 파일을 대상으로

PyPDFLoader를 사용해 문서를 불러옵니다.

실습을 위해, 질의응답에 활용하고 싶은 PDF 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

PDFLoader는 각 페이지를 하나의 Document 단위로 변환하며,

이 단계에서 생성된 문서들은 이후 Text Splitter를 통해 의미 단위로 다시 분할됩니다.

```
In
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("/content/Demian.pdf")
pages = loader.load_and_split()
```

```
In
pages[0]
```

실행 결과

```
Document(metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture Plug-in', 'creator': 'ScanFix(TM) Enhanced',
'creationdate': '2015-09-10T01:40:29+00:00', 'moddate': '2019-01-30T17:47:47+01:00', 'source': '/content/Demian.pdf',
'total_pages': 182, 'page': 0, 'page_label': '1'}, page_content='DEMIAN \n• \nDownloaded from https://www.holybooks.com')
```

```
In
print(pages[10])
```

실행 결과

```
page_content='TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
but we came back half an hour later and fetched them
too.
I hoped for some applause at the end of my story; I
had warmed up to the narrative aJ: last, carried away by
my own eloquence. The two smaller boys were silent,
waiting, Lut Franz Kromer gave me a penetrating look
through his narrow,-1 eyes. "Is that yarn true?" he
asked in a menacing tone.
... (출력 일부 생략)
```

출력 결과를 보기 쉽게 확인하기 위해,

Document 객체 전체가 아닌 실제 텍스트 본문이 담긴 page_content만 선택하여 확인해보겠습니다.

```
In
print(pages[10].page_content)
```

실행 결과

```
TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
but we came back half an hour later and fetched them
too.
I hoped for some applause at the end of my story; I
had warmed up to the narrative aJ: last, carried away by
my own eloquence. The two smaller boys were silent,
waiting, Lut Franz Kromer gave me a penetrating look
through his narrow,-1 eyes. "Is that yarn true?" he
asked in a menacing tone.
... (출력 일부 생략)
```

CSVLoader

SV 파일은 행(row) 단위로 구조화된 데이터를 담고 있는 형식으로,
LangChain의 CSVLoader를 사용하면 각 행을 하나의 Document 객체로 변환할 수 있습니다.
이렇게 변환된 문서들은 이후 PDF나 웹 문서와 동일하게
Embedding, VectorStore, Retrieval 단계에서 함께 활용할 수 있습니다.
실습을 위해, CSV 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

```
In
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("/content/titanic.csv")

data = loader.load()
```

```
In
data[:3]
```

실행 결과

```
[Document(metadata={'source': '/content/titanic.csv', 'row': 0}, page_content='PassengerId: 1\nSurvived: 0\nPclass: 3\nName: Braund, Mr. Owen Harris\nSex: male\nAge: 22\nSibSp: 1\nParch: 0\nTicket: A/5 21171\nFare: 7.25\nCabin: \nEmbarked: S'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 1}, page_content='PassengerId: 2\nSurvived: 1\nPclass: 1\nName: Cumings, Mrs. John Bradley (Florence Briggs Thayer)\nSex: female\nAge: 38\nSibSp: 1\nParch: 0\nTicket: PC 17599\nFare: 71.2833\nCabin: C85\nEmbarked: C'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 2}, page_content='PassengerId: 3\nSurvived: 1\nPclass: 3\nName: Heikkinen, Miss. Laina\nSex: female\nAge: 26\nSibSp: 0\nParch: 0\nTicket: STON/O2. 3101282\nFare: 7.925\nCabin: \nEmbarked: S')]
```

웹베이스로더

웹베이스 로더는 웹페이지에 포함된 텍스트 콘텐츠를 직접 파싱하여 Document 객체로 변환하는 역할을 합니다.
이를 통해 뉴스 기사, 블로그 글, 공지사항과 같은 실시간으로 업데이트되는 웹 문서를 RAG 시스템의 지식 소스로 활용할 수 있습니다.
이번 실습에서는 실제 뉴스 기사를 예제로 사용하여,
웹페이지의 내용을 불러오고 텍스트 형태로 변환하는 과정을 살펴봅니다.

실습에 사용할 웹페이지는 다음과 같습니다.

<https://it.chosun.com/news/articleView.html?idxno=2023092111831>

In

```
from langchain_community.document_loaders import WebBaseLoader
```

실행 결과

```
WARNING: langchain_community.utils.user_agent:USER_AGENT environment variable not set, consider setting it to identify your requests.
```

In

```
loader = WebBaseLoader("https://it.chosun.com/news/articleView.html?idxno=2023092111831")
documents = loader.load()

#print(documents[0].page_content)
```

주석을 해제하고 코드를 실행하면,
해당 웹페이지에 포함된 본문 텍스트 전체를 불러와 확인할 수 있습니다.

웹페이지, PDF, CSV 등 서로 다른 형식의 문서들이
모두 텍스트 형태로 정상적으로 파싱된 것을 확인할 수 있습니다.

이제 이 텍스트를 **전처리(불필요한 요소 제거, 정제)** 한 뒤,
Chunking과 Embedding 단계에 활용할 수 있습니다.

Step2 : TextSplitters 사용해보기

Text Splitter는 긴 텍스트 문서를 **의미를 유지한 작은 단위(Chunk)**로 분할하는 역할을 합니다.
LLM은 한 번에 처리할 수 있는 토큰 수에 제한이 있기 때문에, 문서를 그대로 입력하는 대신 Splitter를 통해 분할된 여러 Chunk를 입력받아 처리하게 됩니다.

이 과정을 통해 긴 문서에서도 토큰 길이 제약을 극복하고, 필요한 부분만 효율적으로 검색할 수 있습니다.

분할된 각 Chunk는 이후 단계에서 1:1로 Embedding되어 VectorStore에 저장되며,
이 Chunk 단위가 RAG 시스템에서 검색과 응답의 기본 단위가 됩니다.

In

```
from langchain_text_splitters import CharacterTextSplitter, RecursiveCharacterTextSplitter
```

CharacterTextSplitter는
하나의 고정된 구분자(separator)를 기준으로 텍스트를 분할하는 방식입니다.
구현이 단순하고 직관적이지만,
문서 구조에 따라 분할된 Chunk가 토큰 제한을 초과하는 경우가 발생할 수 있습니다.

반면, RecursiveCharacterTextSplitter는
줄바꿈, 문장 구분자, 구두점 등 여러 구분자를 순차적으로 적용하며
텍스트를 재귀적으로 분할합니다.

이 방식은 토큰 제한을 안정적으로 만족시키는 데 유리하지만,
분할 과정에서 의미적으로 완전하지 않은 문장 단위로 잘릴 수 있다는 단점이 있습니다.

단순한 구조의 문서나,
문단 구성이 명확한 텍스트의 경우에는 CharacterTextSplitter로도 충분합니다.

하지만 실제 서비스 환경에서는
문서 길이와 구조가 제각각인 경우가 많기 때문에,
대부분의 RAG 시스템에서는 RecursiveCharacterTextSplitter를 기본 선택지로 사용합니다.

이는 Chunk 크기를 안정적으로 제어하면서도
검색 실패를 줄이는 데 유리하기 때문입니다.

```
In
with open("/content/state_of_the_union.txt") as f:
    text = f.read()
```

```
In
#len은 어떤 기준으로 chunk size를 잴 것인가?의 기준이 되는 함수입니다.
#chunk_overlap은 chunk의 앞뒤로 다른 chunk와 설정한 size까지 겹칠 수 있도록 설정하는 것입니다.
text_splitter = CharacterTextSplitter(separator="\n\n", chunk_size=1000, chunk_overlap=100, length_function = len,)
chunks = text_splitter.split_text(text)
```

Chunk의 내용을 확인해보겠습니다

```
In
print(chunks[0])
```

실행 결과

Madam Speaker, Madam Vice President, our First Lady and Second Gentleman. Members of Congress and the Cabinet. Justices of the Supreme Court. My fellow Americans.

Last year COVID-19 kept us apart. This year we are finally together again.

Tonight, we meet as Democrats Republicans and Independents. But most importantly as Americans.

With a duty to one another to the American people to the Constitution.

And with an unwavering resolve that freedom will always triumph over tyranny.

Six days ago, Russia's Vladimir Putin sought to shake the foundations of the free world thinking he could make it bend to his menacing ways. But he badly miscalculated.

He thought he could roll into Ukraine and the world would roll over. Instead he met a wall of strength he never imagined.

He met the Ukrainian people.

From President Zelenskyy to every Ukrainian, their fearlessness, their courage, their determination, inspires the world.

각 chunk의 길이를 확인해보겠습니다,

```
In
length = []
for chunk in chunks:
    length.append(len(chunk))

print(length)
```

실행 결과

[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966, 964, 977, 998, 948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]

토큰 단위로 텍스트 분할해보기

LLM은 문장을 단어가 아닌 토큰(token) 단위로 처리합니다.

따라서 사람이 인식하는 단어 길이나 문자 수는

실제 모델이 처리하는 입력 길이와 정확히 일치하지 않을 수 있습니다.

이로 인해 문자 수나 단어 수를 기준으로 텍스트를 분할할 경우,

모델의 입력 토큰 제한을 초과하거나

예상보다 훨씬 짧은 문맥만 전달되는 문제가 발생할 수 있습니다.

실제 서비스 환경에서는 이러한 문제를 방지하기 위해,

토큰 단위를 기준으로 텍스트를 분할하는 방식을 사용합니다.

이제 토큰 기준으로 텍스트를 분할해보겠습니다.

In

```
!pip install tiktoken
```

실행 결과

(패키지 설치 로그 생략)

In

```
import tiktoken
tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text):
    tokens = tokenizer.encode(
        text
    )
    return len(tokens)
```

In

```
tiktoken_length = []
for chunk in chunks:
    tiktoken_length.append(tiktoken_len(chunk))

print(length)
print(tiktoken_length)
```

실행 결과

```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966, 964, 977, 998,
948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]
[197, 198, 163, 190, 203, 182, 195, 197, 206, 205, 218, 148, 188, 205, 216, 215, 209, 224, 176, 187, 201, 197, 201, 215, 222,
202, 203, 204, 229, 206, 184, 204, 197, 194, 156, 200, 194, 221, 203, 225, 209, 187]
```

글자 수와 토큰 수의 차이를 확인할 수 있습니다 !

Step3 : TextEmbedding 사용해보기

Embedding은 텍스트를 컴퓨터가 계산할 수 있는 수치 벡터(vector) 형태로 변환하는 과정입니다.
이 벡터는 문장의 표면적인 형태가 아니라, 의미적 유사성을 반영하도록 설계되어 있습니다.

변환된 벡터는

VectorStore에 저장되거나,
새로운 질의(Query) 벡터와의 유사도 계산을 통해
의미적으로 가까운 문서를 검색하는 데 사용됩니다.

이러한 변환은 대규모 말뭉치로 사전 학습된
Embedding 전용 모델을 통해 이루어지며,
RAG 시스템에서 Retrieval 성능을 결정하는 핵심 요소입니다.

이번 실습에서는

OpenAI 임베딩 모델을 사용해
텍스트를 벡터로 변환해보겠습니다.

In

```
import openai
```

genai 라이브러리의 list_models 함수를 사용하여 사용 가능한 모델들의 목록을 가져옵니다.

In

```
client = openai.OpenAI()
```

```
In
for model in client.models.list():
    if "embedding" in model.id:
        print(model.id)
```

실행 결과

```
text-embedding-3-small
text-embedding-3-large
text-embedding-ada-002
```

text-embedding-3-small은 가성비가 좋고, text-embedding-3-large는 성능이 더 강력합니다.

```
In
from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

만약 여러분들이 Gemini를 사용하여 구축중이시라면, embedding은 지역에 따라 사용이 제한됩니다.

주로 유럽권에서 제한되기 때문에, 다음 에러를 확인하신다면 Colab 파일의 서버 저장 위치를 확인 후, 다른 임베딩 모델로 변경해야 합니다.

Error embedding content: 400 User location is not supported for the API use.

```
In
#!curl ipinfo.io
```

```
In
# 400 User location is not supported for the API use 오류가 발생한다면, 이 블록을 대신 실행해주세요

# ! pip install -q sentence_transformers

#from langchain.embeddings import HuggingFaceEmbeddings
#embedding_model = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

embedding model 변수에 OpenAI 임베딩모델 혹은 huggingface의 임베딩모델이 할당되었을 것입니다.

embed_documents 멤버 함수를 사용하여 새 문장을 변환해보겠습니다

```
In
embeddings = embedding_model.embed_documents(
    [
        "This is red apple.",
        "This is yellow banana.",
        "This is green lime.",
    ]
)
```

임베딩으로 잘 변환되었는지 확인해보겠습니다

```
In
print(embeddings[1])
```


[0.006435394287109375, -0.0208587646484375, -0.0216064453125, 0.0235595703125, -0.0428466796875, -0.004608154296875, 0.03973388671875, 0.05267333984375, -0.0018892288208007812, -0.0237579345703125, 0.0137939453125, 0.0159912109375, -0.044464111328125, -0.00013935565948486328, -0.007648468017578125, 0.02557373046875, 0.0254058837890625, 0.035491943359375, -0.0517578125, 0.01232147216796875, 0.0248260498046875, -0.00045800209045410156, 0.0193939208984375, 0.07525634765625, -0.0020503997802734375, -0.00921630859375, 0.016815185546875, -0.007152557373046875, 0.02655029296875, -0.048492431640625, 0.043182373046875, -0.04254150390625, 0.01165008544921875, -0.032867431640625, -0.0333251953125, -0.046142578125, 0.0019063949584960938, 0.0225677490234375, -0.01812744140625, 0.03253173828125, 0.0293121337890625, 0.011749267578125, -0.01442718505859375, 0.0030574798583984375, 0.0243377685546875, 0.048187255859375, -0.01355743408203125, 0.049346923828125, -0.040618896484375, 0.04400634765625, 0.04522705078125, 0.0195770263671875, 0.0203094482421875, 0.00183868408203125, 0.0183258056640625, -0.0193939208984375, 0.030426025390625, 0.04461669921875, -0.030853271484375, -0.000415802001953125, -0.00917816162109375, 0.005367279052734375, -0.06231689453125, 0.053009033203125, -0.001255035400390625, -0.0166473388671875, -0.04693603515625, -0.0004734992980957031, 6.276369094848633e-05, 0.0068511962890625, -0.01000213623046875, 0.03692626953125, -0.025115966796875, -0.019622802734375, 0.01117706298828125, 0.05419921875, 0.0145263671875, -0.004810333251953125, -0.02996826171875, 0.033050537109375, -0.013214111328125, 0.0017747879028320312, -0.027191162109375, -0.0284881591796875, 0.03131103515625, -0.005939483642578125, -0.052886962890625, -0.034149169921875, 0.0268402099609375, -0.001132965087890625, -0.033111572265625, -0.019195556640625, -0.00717926025390625, 0.01003265380859375, 0.0026149749755859375, 0.001748046875, -0.018890380859375, 0.001613616943359375, 0.0003464221954345703, -0.0251312255859375, 0.0013561248779296875, 0.0008759498596191406, -0.0011882781982421875, 0.029937744140625, 0.053985595703125, -0.005321502685546875, 0.00035500526428222656, -0.0010051727294921875, -0.033050537109375, 0.01355743408203125, 0.0036163330078125, 0.0252685546875, 0.061370849609375, 0.0229949951171875, -0.0267486572265625, 0.0156402587890625, 0.0127410888671875, -0.04815673828125, 0.04437255859375, -0.10479736328125, 0.029449462890625, 0.007434844970703125, -0.003971099853515625, -0.062042236328125, -0.08038330078125, -0.0032520294189453125, -0.010986328125, 0.042816162109375, 0.01000213623046875, -0.0192718505859375, 0.0440673828125, -0.0177154541015625, -0.051177978515625, 0.05157470703125, -0.0002944469451904297, 0.020751953125, -0.043731689453125, -0.0111236572265625, -0.009613037109375, -0.01885986328125, 0.0232696533203125, -0.00292205810546875, -0.01534271240234375, -0.04010009765625, 0.032440185546875, 0.00748443603515625, -0.00878143310546875, 0.00829315185546875, -0.02447509765625, 0.0087432861328125, 0.055633544921875, 0.01552581787109375, -0.055145263671875, 0.00701141357421875, -0.048187255859375, -0.00951385498046875, 0.07275390625, -0.01519775390625, 0.0487060546875, 0.06005859375, -0.0040435791015625, -0.072265625, -0.00434112548828125, 0.0111236572265625, 0.0021533966064453125, 0.037384033203125, -0.0389404296875, -0.0163421630859375, -0.039825439453125, 0.005764007568359375, 0.013946533203125, 0.016571044921875, 0.0229034423828125, 0.0013103485107421875, 0.006481170654296875, -0.0003402233123779297, -0.00524139404296875, -0.029449462890625, -0.0272979736328125, 0.004852294921875, 0.032318115234375, 0.04925537109375, -0.0174407958984375, -0.03765869140625, -0.03729248046875, -0.037384033203125, 0.0182037353515625, 0.022125244140625, -0.0009660720825195312, -0.0144805908203125, 0.0204010009765625, 0.0207366943359375, 0.0027256011962890625, 0.04119873046875, -0.004077911376953125, -0.035675048828125, 0.0389404296875, 0.00893402099609375, 0.0013341903686523438, -0.0018262863159179688, -0.0217437744140625, 0.049224853515625, -0.0192108154296875, 0.0270843505859375, -0.0166473388671875, -0.0164794921875, 0.003757476806640625, 0.05169677734375, 0.01358795166015625, 0.0235443115234375, 0.049774169921875, 0.04266357421875, 0.0287017822265625, -0.0255126953125, -0.042266845703125, -0.0164642333984375, -0.0028705596923828125, 0.043487548828125, -0.01152801513671875, 0.0086669921875, 0.004276275634765625, 0.0006494522094726562, 0.07684326171875, 0.03411865234375, -0.05670166015625, -0.0012083053588867188, 0.028045654296875, 0.012481689453125, -0.024505615234375, -0.0210418701171875, 0.0300750732421875, -0.00493621826171875, -0.021087646484375, -0.027801513671875, -0.0183868408203125, -0.003269195556640625, -0.02099609375, 0.04266357421875, 0.0289154052734375, -0.006809234619140625, 0.0038776397705078125, 0.0178985595703125, 0.04364013671875, 0.039276123046875, 0.049957275390625, -0.023193359375, -0.03887939453125, -0.0180816650390625, 0.0218963623046875, 0.05029296875, 0.0039520263671875, 0.0278167724609375, -0.027130126953125, -0.0404052734375, -0.025390625, -0.0310821533203125, -0.068603515625, -0.019256591796875, 0.033782958984375, -0.01090240478515625, 0.005710601806640625, -0.03326416015625, -0.024932861328125, -0.032928466796875, -0.0181884765625, -0.0147552490234375, 0.0557861328125, -0.01122283935546875, -0.022705078125, 0.0018672943115234375, -0.0185394287109375, -0.040374755859375, 0.0511474609375, 0.00576019287109375, -0.04296875, -0.02313232421875, -0.0009241104125976562, -0.0025653839111328125, -0.036102294921875, 0.00772857666015625, 0.00939178466796875, -0.008331298828125, 0.00473785400390625, -0.00824737548828125, 0.0216827392578125, -0.039276123046875, 0.0305023193359375, -0.01007843017578125, 0.004169464111328125, 0.010498046875, 0.02825927734375, -0.0175018310546875, 0.053497314453125, -0.0202178955078125, 0.005558013916015625, 0.01480865478515625, 0.0015096664428710938, 0.0294036865234375, -0.006214141845703125, -0.037994384765625, -0.037994384765625, 0.018218994140625, -0.0234832763671875, 0.0036296844482421875, -0.03216552734375, 0.0184783935546875, -0.02996826171875, 0.0231781005859375, -0.03155517578125, -0.038848876953125, -0.02685546875, 0.004619598388671875, 0.040771484375, -0.040008544921875, 0.03826904296875, -0.04144287109375, 0.0043792724609375, 0.02288818359375, 0.0088958740234375, 0.0178375244140625, 0.02459716796875, 0.01031494140625, -0.00379180908203125, -0.0076904296875, 0.0013284683227539062, 0.033599853515625, 0.00917816162109375, 0.0189361572265625, -0.01806640625, 0.0078125, -0.0423583984375, -0.037353515625, 0.0160369873046875, -0.007598876953125, -0.039154052734375, -0.022320556640625, -0.0291290283203125, -0.0404052734375, -0.00302886962890625, 0.0164031982421875, -0.0055999755859375, -0.04827880859375, 0.07861328125, -7.18832015991211e-05, -0.047515869140625, -0.0296478271484375, 0.05078125, 0.005062103271484375, -0.061370849609375, -0.05731201171875, -0.020808056640625, 0.009124755859375, 0.0028171539306640625, -0.0186309814453125, 0.015167236328125, -0.0210418701171875, -0.01433563232421875, -0.01508331298828125, -0.029998779296875, 0.006710052490234375, -0.0007948875427246094, 0.0221405029296875, 0.023040771484375, 0.03564453125, 0.045166015625, 0.0109710693359375, -0.03509521484375, -0.07171630859375, 0.06317138671875, 0.00225830078125, -0.062225341796875, -0.0244903564453125, -0.004474639892578125, 0.0214996337890625, -0.0071563720703125, -0.04351806640625, 0.03631591796875, 0.010040283203125, 0.024383544921875, -0.050048828125, 0.006801605224609375, 0.02685546875, 0.01708984375, 0.01062774658203125, 0.01139068603515625, -0.048614501953125, 0.0037059783935546875, -0.003574371337890625, -0.0396728515625, -0.030029296875, 0.0218048095703125, -0.0289459228515625, 0.01081085205078125, -0.041473388671875, -0.00043463706970214844, 0.048126220703125, 0.0017547607421875, -0.0032672882080078125,

-0.047576904296875, 0.045623779296875, 0.041778564453125, -0.032196044921875, -0.05206298828125, 0.0037212371826171875, -0.0076751708984375, 0.006900787353515625, -0.0173187255859375, -0.037384033203125, 0.0093536376953125, -0.01519775390625, 0.0264129638671875, -0.043487548828125, 0.0195465807890625, -0.002685546875, 0.038299560546875, -0.03619384765625, 0.0198974609375, 0.033477783203125, -0.0143585205078125, 0.004039764404296875, 0.01132965087890625, 0.0185699462890625, -0.02375179345703125, -0.01049041748046875, -0.0201263427734375, 0.058746337890625, -0.0227203369140625, -0.07177734375, 0.0165252685546875, -0.038787841796875, 0.056427001953125, -0.0189056396484375, -0.0223541259765625, 0.0171356201171875, -0.0220489501953125, -0.041595458984375, -0.0129241943359375, 0.020538330078125, -0.03460693359375, -0.034088134765625, 0.02191162109375, -0.03277587890625, 0.0252532958984375, 0.0021724700927734375, -0.034088134765625, 0.031280517578125, 0.054779052734375, 0.0181732177734375, -0.020416259765625, -0.024078369140625, -0.00118255615234375, -0.025665283203125, -0.10406494140625, 0.026580810546875, -0.072021484375, 0.0232696533203125, -0.018218994140625, 0.05242919921875, 0.01461029052734375, -0.01220703125, 0.0290069580078125, 0.003841400146484375, 0.003787994384765625, 0.0264892578125, 0.018402099609375, -0.0229034423828125, -0.01739501953125, 0.033843994140625, 0.0675048828125, 0.01910400390625, 0.0172119140625, 0.0091094970703125, 0.04400634765625, 0.0718994140625, -0.01024627685546875, 0.026763916015625, 0.0379638671875, -0.037567138671875, 0.00510406494140625, -0.0068511962890625, -0.0027599334716796875, 0.03924560546875, -0.034576416015625, -0.0408935546875, 0.000640869140625, 0.01264190673828125, 0.03240966796875, 0.00914764404296875, -0.030792236328125, 0.01012420654296875, -0.02740478515625, -0.0014753341674804688, -0.0241546630859375, -0.0161285400390625, 0.034942626953125, 0.004726409912109375, 0.0283966064453125, -0.0633544921875, -0.00717926025390625, -0.0200958251953125, -0.010040283203125, -0.03277587890625, 0.0021800994873046875, -0.0172271728515625, 0.0296173095703125, -0.037017822265625, -0.01325225830078125, 0.0041351318359375, -0.00861358642578125, -3.886222839355469e-05, -0.029144287109375, 0.0025691986083984375, -0.0270843505859375, -0.0609130859375, -0.066650390625, 0.009033203125, 0.0267486572265625, 0.002105712890625, -0.002226905517578125, -0.005863189697265625, 0.00594329833984375, 0.004802703857421875, -0.01029205322265625, -0.026153564453125, -0.016021728515625, 0.0194244384765625, 0.0245361328125, -0.042633056640625, -0.0197601318359375, 0.00405120849609375, 0.0111846923828125, -0.04443359375, -0.01361846923828125, -0.032928466796875, -0.005107879638671875, 0.0020313262939453125, -0.00267791748046875, -0.0004279613494873047, 0.03839111328125, 0.0239105224609375, 0.00015079975128173828, -0.0009851455688476562, 0.0272979736328125, -0.021026611328125, -0.0291900634765625, -0.0095367431640625, 0.0019245147705078125, 0.0167236328125, -0.01165008544921875, 0.0496826171875, 0.0008945465087890625, -0.017578125, 0.002941131591796875, 0.004055023193359375, -0.0078582763671875, 0.0198211669921875, -0.0091094970703125, -0.007648468017578125, -0.01166534423828125, 0.034759521484375, 0.030853271484375, -0.008682509765625, -0.0181427001953125, -0.0056610107421875, 0.019805908203125, 0.005767822265625, -0.00406646728515625, 0.0077056884765625, -0.02606201171875, 0.0247039794921875, -0.046234130859375, 0.02471923828125, -0.019134521484375, -0.0643310546875, -0.005794525146484375, 0.0269012451171875, 0.038055419921875, 0.00627899169921875, 0.039459228515625, -0.00838470458984375, -0.01186370849609375, -0.0494384765625, -0.0196380615234375, -0.023712158203125, -0.0145111083984375, -0.01348876953125, -0.018096923828125, -0.03387451171875, -0.0350341796875, -0.015380859375, 0.0174713134765625, -0.0227203369140625, 0.023651123046875, -0.0185699462890625, 0.005474090576171875, -0.03973388671875, -0.027069091796875, -0.00907135009765625, -0.009857177734375, 0.02288818359375, 0.0203704833984375, 0.0231781005859375, 0.0362548828125, -0.036865234375, 0.00357818603515625, 0.050506591796875, -0.0308837890625, -0.01351165771484375, 0.0213623046875, -0.01332855224609375, -0.02593994140625, 0.032440185546875, 0.01268768310546875, -0.01503753662109375, 0.03515625, -0.00202178955078125, 0.004238128662109375, -0.00545501708984375, -0.019012451171875, 0.0014095306396484375, 0.033050537109375, -0.0112457275390625, 0.0113525390625, -0.036956787109375, -0.0263519287109375, 0.045654296875, 0.048248291015625, -0.0027675628662109375, 0.0167999267578125, -0.0309906005859375, -0.01209259033203125, -0.032623291015625, 0.02679443359375, -0.00931549072265625, -0.0185089111328125, 0.0188140869140625, 0.0147857666015625, -0.0272369384765625, 0.0027980804443359375, -0.0173187255859375, 0.005786895751953125, 0.0211944580078125, -0.0219879150390625, -0.02154541015625, 0.023406982421875, -0.034393310546875, 0.0010309219360351562, -0.015655517578125, -0.0162811279296875, -0.003963470458984375, 0.0055694580078125, -0.007564544677734375, 0.00873565673828125, -0.027587890625, -0.0279083251953125, -0.0065460205078125, -0.00946044921875, 0.0042877197265625, -0.022369384765625, -0.00946807861328125, 0.00960540771484375, 0.045745849609375, 0.01055145263671875, -0.00913238525390625, 0.00482177734375, -0.022552490234375, 0.0027484893798828125, -0.0013895034790039062, 0.05792236328125, -0.03887939453125, 0.0302734375, -0.0219879150390625, -0.0093536376953125, -0.044525146484375, 0.0367431640625, 0.0269622802734375, -0.009124755859375, -0.00255584716796875, -0.00397491455078125, 0.01383209228515625, -0.026092529296875, -0.032470703125, -0.024017333984375, 0.0095977783203125, 0.049835205078125, 0.0021610260009765625, 0.035247802734375, 0.0384521484375, -0.022125244140625, -0.03778076171875, -0.01885986328125, 0.044158935546875, 0.0150299072265625, 0.008514404296875, -0.022125244140625, 0.025726318359375, -0.00211334228515625, -0.003795623779296875, 0.0037746429443359375, -0.011749267578125, -0.0048828125, 0.00768280029296875, -0.04754638671875, 0.012847900390625, -0.04632568359375, 0.0039520263671875, 0.00759124755859375, 0.0491943359375, 0.0545654296875, 0.0270843505859375, -0.0114898681640625, -0.023284912109375, -0.06097412109375, 0.043701171875, 0.01331329345703125, -0.01373291015625, -0.05523681640625, -0.009185791015625, 0.006130218505859375, 0.01506805419921875, 0.00609588623046875, 0.030792236328125, -0.019500732421875, -0.01334381103515625, 0.01433563232421875, 0.01195526123046875, 0.01739501953125, -0.01395416259765625, -0.0063323974609375, -0.006465911865234375, 0.011016845703125, 0.033203125, -0.01309967041015625, 0.0176239013671875, -0.001361846923828125, -0.01459503173828125, 0.01509857177734375, 0.01436614990234375, -0.00838470458984375, -0.04638671875, -0.0012960433959960938, 0.022125244140625, -0.0133056640625, -0.0244293212890625, -0.006092071533203125, 0.0022640228271484375, 0.0165863037109375, 8.171796798706055e-05, -0.015838623046875, 0.024566650390625, -0.016845703125, 0.013031005859375, -0.006259918212890625, 0.042694091796875, -0.00522613525390625, -0.01318359375, 0.0211029052734375, -0.0175018310546875, -0.006000518798828125, 0.035186767578125, -0.04144287109375, 0.02557373046875, 0.0322265625, 0.007541656494140625, 0.003177642822265625, 0.02197265625, 0.01554107666015625, -0.031402587890625, 0.027099609375, 0.00446319580078125, -0.0119171142578125, -0.0296173095703125, 0.0153656005859375, 0.016571044921875, 0.0421142578125, 0.00762176513671875, -0.0026187896728515625, -0.020559365234375, -0.01192474365234375, 0.0208892822265625, -0.032440185546875, -0.006038665771484375, 0.00014328956604003906, -0.0059661865234375, 0.0161285400390625, -0.0285797119140625, -0.01177978515625, -0.0201263427734375, -0.064208984375, -0.005344390869140625, -0.01019287109375, -0.00629425048828125, -0.007598876953125, -0.031524658203125, 0.025543212890625, 0.0022525787353515625, 0.0006079673767089844, -0.054718017578125, -0.02960205078125, 0.0161895751953125, 0.0172119140625, 0.0184173583984375,

0.0151824951171875, 0.0037822723388671875, 0.0019779205322265625, -0.020263671875, -0.00414276123046875, 0.0047760009765625, -0.0037841796875, 0.034820556640625, 0.0159149169921875, 0.0076904296875, -0.031494140625, -0.013519287109375, 0.006198883056640625, -0.005542755126953125, -0.006267547607421875, -0.05242919921875, -0.0199127197265625, 0.006256103515625, 0.0110015869140625, 0.007526397705078125, -0.0251617431640625, -0.0109405517578125, -0.0112152099609375, 0.035888671875, -0.015899658203125, 0.037994384765625, -0.015167236328125, -0.0031337738037109375, 0.033111572265625, -0.03759765625, 0.0249481201171875, -0.0032958984375, -0.01004791259765625, 0.0242462158203125, -0.01427459716796875, -0.0278778076171875, -0.032135009765625, -0.007297515869140625, 0.007541656494140625, 0.02703857421875, -0.00675201416015625, 0.0230712890625, 0.007266998291015625, -0.0165863037109375, -0.0195465087890625, -0.006072998046875, -0.01300811767578125, 0.0170745849609375, -0.018341064453125, -0.036773681640625, 0.018310546875, 0.00855255126953125, 0.036163330078125, -0.0217742919921875, 0.0145111083984375, 0.023345947265625, -0.03729248046875, 0.014801025390625, 0.0069427490234375, 0.0369873046875, 0.0038280487060546875, 0.00189208984375, 0.022003173828125, 0.0100860595703125, -0.0509033203125, 0.00812530517578125, 0.0011873245239257812, 0.0340576171875, 0.0178375244140625, -0.023895263671875, 0.020721435546875, 0.00487518310546875, 0.026580810546875, 0.01824951171875, -0.0244140625, -0.008819580078125, -0.002780914306640625, -0.0362548828125, -0.018508911328125, -0.01227569580078125, -0.0003063678741455078, -0.0161590576171875, -0.027740478515625, -0.0085601806640625, 0.0218353271484375, 0.0318603515625, 0.030914306640625, 0.004730224609375, -0.01023101806640625, -0.0413818359375, 0.007732391357421875, -0.006153106689453125, -0.007175445556640625, 0.01812744140625, 0.020233154296875, -0.00146484375, 0.035247802734375, -0.013824462890625, -0.020416259765625, 0.01012420654296875, -0.02093505859375, 0.03900146484375, 0.00626373291015625, -0.01288604736328125, 0.024810791015625, -0.026580810546875, 0.0057220458984375, 0.0174713134765625, -0.0193328857421875, -0.02484130859375, 0.06707763671875, -0.01617431640625, 0.0250091552734375, -0.00991058349609375, -0.0097503662109375, 0.008544921875, -0.01381683349609375, -0.00440216064453125, 0.006267547607421875, -0.02825927734375, -0.02056884765625, 0.009063720703125, -0.0234832763671875, -0.0036163330078125, -0.0364798828125, -0.00905609130859375, -0.0102081298828125, 0.024200439453125, -0.009979248046875, 0.01091766357421875, -0.0166778564453125, -0.01312255859375, 9.876489639282227e-05, -0.0199432373046875, 0.00882720947265625, 0.01593017578125, 0.014129638671875, -0.03076171875, 0.006198883056640625, -0.040771484375, 0.041229248046875, -0.039825439453125, 0.006282806396484375, 0.0115814208984375, -0.001186370849609375, -0.011962890625, 0.00801849365234375, -0.0294036865234375, 0.005794525146484375, 0.00804901123046875, 0.0299072265625, -0.03662109375, 0.0065765380859375, 0.0282745361328125, 0.046417236328125, 0.0033702850341796875, 0.018157958984375, 0.0166015625, -0.027374267578125, -0.0108184814453125, 0.0216217041015625, -0.0116729736328125, 0.026885986328125, 0.0025577545166015625, 0.045379638671875, 0.0214691162109375, 0.0557861328125, 0.006618499755859375, -0.005886077880859375, -0.0124969482421875, 0.028656005859375, -0.006259918212890625, -0.010772705078125, 0.005146026611328125, 0.05999755859375, -0.048095703125, 0.0302581787109375, -0.044403076171875, 0.0297088623046875, 0.034515380859375, 0.0005111694359375, 0.021331787109375, -0.03155517578125, -0.0215301513671875, 0.030487060546875, -0.013702392578125, -0.01004791259765625, -0.0128021240234375, -0.035797119140625, -0.00223541259765625, -0.01409149169921875, 0.0280303955078125, 0.0275726318359375, 0.0159759521484375, 0.0207061767578125, 0.0968017578125, -0.008026123046875, -0.009002685546875, 0.0190277099609375, 0.01111602783203125, 0.0060043349609375, -0.01264190673828125, 0.003620147705078125, -0.0293426513671875, 0.0270538330078125, 0.0023040771484375, -0.004119873046875, 0.0026092529296875, -0.037506103515625, -0.0154266357421875, 0.01000213623046875, -0.042327880859375, 0.0215606689453125, -0.01267242431640625, 0.0162506103515625, -0.01378631591796875, 0.0032749176025390625, 0.0154266357421875, 0.0201416015625, -0.01226806640625, -0.016571044921875, -0.016510009765625, 0.00786590576171875, -0.0267333984375, 0.005313873291015625, -0.0201873779296875, 0.0144805908203125, -0.01262664794921875, -0.0114593505859375, -0.0223541259765625, -0.035491943359375, 0.0293121337890625, 0.005191802978515625, 0.025115966796875, 0.01552581787109375, -0.0002415180206298828, -0.0235443115234375, -0.00893402099609375, -0.0026111602783203125, -0.047088623046875, -0.002590179443359375, -0.0018510818481445312, 0.00981903076171875, -0.025390625, -0.004802703857421875, -0.01270294189453125, -0.038482666015625, -0.0017843246459960938, -0.0379638671875, -0.01898193359375, -0.0261688232421875, -0.020050048828125, 0.0178985595703125, -0.0055419921875, 0.0235443115234375, 0.004222869873046875, 0.00234222412109375, 0.018341064453125, -0.0293121337890625, -0.01806640625, 0.00384521484375, 0.0022182464599609375, 0.01213836669921875, 0.004421234130859375, 0.043701171875, 0.019927978515625, 0.0267791748046875, 0.0176849365234375, -0.01039886474609375, 0.0010166168212890625, 0.00846099853515625, 0.02203369140625, -0.01076507568359375, 0.0269622802734375, -0.0006136894226074219, -0.005870819091796875, 0.0032749176025390625, 0.0202178955078125, 0.04992677818125, 0.0133056640625, -0.003414154052734375, -0.01544189453125, -0.041473388671875, -0.0141754150390625, 0.041015625, 0.02532958984375, 0.0183563232421875, 0.01082611083984375, -0.0004467964172363281, 0.0010957717895507812, 0.0149383544921875, 0.037872314453125, -0.0160064697265625, 0.0015125274658203125, 0.0106201171875, 0.0301055908203125, 0.01483917236328125, -0.0029430389404296875, -0.01800537109375, 0.0004532337188720703, 0.0137786865234375, 0.043548583984375, 0.0055999755859375, -0.0089111328125, 0.0305633544921875, 0.026153564453125, 0.004375457763671875, -0.004474639892578125, 0.0296783447265625, -0.01593017578125, -0.014434814453125, 0.04107666015625, -0.0239410400390625, 0.007518768310546875, -0.0185546875, 0.00494384765625, -0.028472900390625, -0.0045318603515625, -0.0007386207580566406, 0.0113983154296875, 0.0428466796875, 0.00955963134765625, 0.0174981689453125, -0.0123443603515625, 0.02386474609375, -0.04534912109375, -0.01386260986328125, -0.0220184326171875, 0.0292816162109375, -0.0033779144287109375, -0.0164642333984375, -0.0014657974243164062, -0.0027751922607421875, -0.01113128662109375, -0.00933074951171875, 0.0174407958984375, -0.0207366943359375, -0.01593017578125, 0.0286407470703125, 0.0186920166015625, 0.01505279541015625, 0.0256195068359375, 0.011077880859375, 0.018798828125, 0.01116943359375, 0.11517333984375, -0.027862548828125, 0.006641387939453125, -0.01422119140625, 0.005886077880859375, -0.0184783935546875, -0.0308837890625, 0.025146484375, -0.01251983642578125, 0.0245361328125, 0.01470184326171875, 0.00890350341796875, -0.0173797607421875, -0.027587890625, 0.037109375, 0.050201416015625, -0.036712646484375, -0.023223876953125, -0.003795623779296875, -0.0111236572265625, 0.007190704345703125, 0.00267791748046875, -0.007183074951171875, -0.005466461181640625, 0.00032401084899902344, -0.006320953369140625, -0.006900787353515625, 0.00420379638671875, -0.03692626953125, 0.014739990234375, 0.00783538818359375, 0.016571044921875, -0.004150390625, -0.005313873291015625, -0.00920867919921875, 0.034454345703125, -0.012176513671875, -0.0277099609375, 0.043212890625, -0.031951904296875, 0.02783203125, -0.0081787109375, 0.00521087646484375, 0.031982421875, -0.0117034912109375, 0.027252197265625, -0.0230255126953125, 0.00954437255859375, 0.01534271240234375, -0.007160186767578125, 0.04583740234375, -0.006938934326171875, -0.01332855224609375, 0.05303955078125, -0.000762939453125, -0.0019369125366210938, -0.0193328857421875, 0.0080718994140625, -0.01090240478515625, 0.00490570068359375, -0.01383209228515625,

0.005992889404296875, 0.024078369140625, 0.044952392578125, 0.0254364013671875, 0.01195526123046875, -0.0301666259765625, 0.01355743408203125, -0.00014770030975341797, 0.00968170166015625, 0.015594482421875, 0.0178070068359375, -0.00905609130859375, 0.003803253173828125, 0.036651611328125, 0.0305328369140625, 0.0157623291015625, -0.0172882080078125, 0.0277099609375, 0.050994873046875, 0.031280517578125, -0.03173828125, 0.034393310546875, -0.0111236572265625, 0.01486968994140625, -0.00997161865234375, -0.01468658447265625, 0.047271728515625, 0.0135040283203125, -0.0207061767578125, 0.0225067138671875, -0.0247344970703125, 0.0014181137084960938, -0.0011157989501953125, -0.0014867782592773438, -0.0069427490234375, -0.01151275634765625, -0.0024051666259765625, 0.01485443115234375, 0.005786895751953125, 0.00830841064453125, 0.0125732421875, -0.008514404296875, -0.021392822265625, -0.0172576904296875, -0.0176239013671875, 0.0033855438232421875, 0.016571044921875, 0.001392364501953125, -0.00399017333984375, -0.021942138671875, 0.021575927734375, 0.03643798828125, -0.0197601318359375, 0.006542205810546875, -0.005023956298828125, -0.0121002197265625, -0.022796630859375, -0.059417724609375, 0.0120086669921875, 0.00023186206817626953, 0.039794921875, -0.0010309219360351562, -0.0010519027709960938, 0.01678466796875, -0.040283203125, 0.01508331298828125, -0.00965118408203125, -0.0134429931640625, -0.015777587890625, 0.0192108154296875, 0.01094818115234375, 0.038818359375, 0.00919342041015625, 0.03607177734375, -0.004764556884765625, -0.040618896484375, -0.0285797119140625, -0.0147247314453125, 0.0263519287109375, 0.0184783935546875, 0.023712158203125, 0.0158233642578125, -0.021881103515625, 0.028656005859375, 0.01528167724609375, 0.0167999267578125, -0.0002932548522949219, 0.00817108154296875, 0.02520751953125, 0.01419830322265625, 0.00800323486328125, 0.0011568069458007812, -0.01361083984375, -0.01129150390625, -0.00595855712890625, 0.0038280487060546875, 0.00981903076171875, -0.0276641845703125, 0.00347900390625, 0.0101470947265625, 0.0006623268127441406, -0.00083160400390625, 0.0240631103515625, -0.0020961761474609375, 0.0196380615234375, -0.01483917236328125, 0.00931549072265625, 0.01373291015625, -0.001964569091796875, -0.0013628005981445312, 0.01349639892578125, -0.01153564453125, 0.0404052734375, -0.01007843017578125, -0.0225982666015625, 0.006725311279296875, 0.054229736328125, -0.00334930419921875, 0.0181427001953125, -0.0655517578125, -0.01261138916015625, 0.02685546875, -0.0124053955078125, -0.026580810546875, 0.01442718505859375, -0.006053924560546875, -0.033843994140625, 0.01513671875, -0.0188140869140625, 0.04608154296875, -0.0065765380859375, -0.02484130859375, 0.0321044921875, -0.0225372314453125, 0.00670623779296875, -0.0384521484375, 0.0123748779296875, 0.03497314453125, 0.0362548828125, 0.0149383544921875, 0.00865936279296875, -0.0182952880859375, 0.0302581787109375, 0.02587890625, 0.011627197265625, -0.0122222900390625, 0.00997161865234375, -0.010711669921875, -0.01294708251953125, 0.0180816650390625, 0.01416015625, -0.0275421142578125, 0.0094451904296875, -0.00390625, 0.0048370361328125, -0.01436614990234375, -0.012420654296875, -0.033599853515625, 0.025726318359375, -0.032196044921875, -0.0100555419921875, 0.004001617431640625, -0.02288818359375, -0.0226898193359375, 0.018280029296875, -0.0116119384765625, -0.0160980224609375, -0.0152130126953125, -0.0260467529296875, 0.0276031494140625, -0.016448974609375, -0.006526947021484375, -0.01812744140625, 0.00873565673828125, 0.0008091926574707031, 0.01666259765625, 0.0152435302734375, -0.02313232421875, -0.042724609375, -0.032012939453125, 0.008575439453125, -0.015899658203125, 0.0126800537109375, -0.027099609375, 0.0118865966796875, 0.012664794921875, -0.02679443359375, -0.0247650146484375, 0.01490020751953125, 0.01419830322265625, 0.0224456787109375, 0.022979736328125, -0.0204315185546875, -0.004482269287109375, -0.03173828125, 0.0297088623046875, -0.0296173095703125, -0.00318145751953125, 0.0010786056518554688, -0.04248046875, -0.0243072509765625, -0.0023975372314453125, 0.00714111328125, -0.0099639892578125, -0.018829345703125, -0.001922607421875, -0.00959014892578125, 0.0007562637329101562, 0.00449371337890625, 0.009063720703125, -0.01239776611328125, 0.00626373291015625, -0.023345947265625, -0.004138946533203125, 0.009002685546875, 0.003993988037109375, -0.005584716796875, -0.0160980224609375, -0.0223236083984375, 0.0271759033203125, -0.03045654296875, -0.01462554931640625, -0.03753662109375, -0.0430908203125, 0.006526947021484375, -0.0035686492919921875, 0.007312774658203125, -0.05291748046875, 0.0007224082946777344, -0.0147857666015625, 0.008087158203125, 0.02001953125, -0.009246826171875, -0.01690673828125, -0.00492095947265625, 0.04986572265625, 0.01508331298828125, 0.0135040283203125, 0.0035495758056640625, -0.02874755859375, -0.017852783203125, -0.01343536376953125, -0.0179901123046875, -0.028533935546875, 0.01387786865234375, -0.00603485107421875, 0.01557159423828125, 0.0260772705078125, -0.0175323486328125, 0.053375244140625, 0.0100555419921875, -0.0117034912109375, 0.0010700225830078125, 0.0056915283203125, -0.0020160675048828125, 0.0345458984375, -0.02060991796875, -0.05023193359375, 0.017822265625, 0.06060791015625, 0.019378662109375, -0.01418304443359375, 0.01360321044921875, 0.0245513916015625, -0.012664794921875, 0.007411956787109375, 0.02001953125, 0.015228271484375, -0.02862548828125, 0.034912109375, -0.00926971435546875, 0.003894805908203125, 0.0217132568359375, 0.0236968994140625, -0.015625, -0.0292205810546875, -0.012451171875, -0.00778961181640625, -0.006778717041015625, -0.0292205810546875, -0.0226593017578125, -0.002574920654296875, -0.01165771484375, 0.0094451904296875, 0.03363037109375, 0.00452423095703125, 0.0233306884765625, -0.028228759765625, -0.00478363037109375, 0.00786590576171875, 0.0185089111328125, -0.011962890625, 0.0080413818359375, 0.038299560546875, 0.01580810546875, 0.01143646240234375, 0.0035610198974609375, 0.0232696533203125, 0.0029850006103515625, -0.0283660888671875, -0.03619384765625, 0.033172607421875, -0.03619384765625, 0.007778167724609375, -0.0012865066528320312, -0.021270751953125, 0.0202484130859375, 0.01441192626953125, 0.00543212890625, -0.0238189697265625, 0.0311431884765625, 0.0172271728515625, 0.004535675048828125, 0.0382080078125, 0.0171966552734375, 0.002780914306640625, 0.0174407958984375, -0.0265045166015625, -0.0102081298828125, -0.0028362274169921875, 0.00615692138671875, -0.0269927978515625, 0.005123138427734375, 0.0081634521484375, 0.01519775390625, -0.0031375885009765625, -0.01372528076171875, -0.00441741943359375]

In

len(embeddings[1])

실행 결과

1536

새로운 쿼리를 넣어, 임베딩끼리 유사도를 계산해보겠습니다

```
In
import numpy as np
from numpy import dot
from numpy.linalg import norm
def cos_sim(A, B):
    return dot(A, B)/(norm(A)*norm(B))
```

```
In
query = ["this is red fruit"]
```

```
In
e_query = embedding_model.embed_documents(query)
print(cos_sim(embeddings[0], e_query[0]))
print(cos_sim(embeddings[1], e_query[0]))
print(cos_sim(embeddings[2], e_query[0]))
```

실행 결과

```
0.7477942151308524
0.48995458116791124
0.4084112226829356
```

빨간 사과와 빨간 과일의 유사도가 많이 높게 나왔습니다!

임베딩 모델은 사용 언어나 필요에 따라 다양하게 교체하여 사용할 수 있습니다.

해당 링크에서 여러 목록을 확인하실 수 있습니다.

https://python.langchain.com/docs/integrations/text_embedding/

Step4 : VectorStore 사용해보기

VectorStore는 텍스트를 Embedding 모델을 통해 벡터(vector)로 변환한 뒤, 이를 저장하고 관리하는 저장소입니다.

이 저장소는 단순한 데이터 보관 공간이 아니라,

벡터 간의 유사도를 빠르게 계산하고 탐색하기 위한 인덱싱 구조를 함께 포함하고 있습니다.

문서나 쿼리가 Embedding된 이후에는,

VectorStore를 통해 의미적으로 유사한 벡터를 효율적으로 검색할 수 있으며,

이 과정이 RAG 시스템의 Retrieval 단계를 담당하게 됩니다.

대표적인 VectorStore로는

Chroma, FAISS 등이 있으며,

각각 로컬 환경과 대규모 서비스 환경에서 널리 사용됩니다.

이번 실습에서는

구성이 단순하고 로컬 환경에서 바로 사용할 수 있는

ChromaDB를 사용해 VectorStore를 구성해보겠습니다.

```
In
!pip install chromadb
```

실행 결과

```
(패키지 설치 로그 생략)
```

```
In
!pip install langchain-chroma
```

실행 결과

```
(패키지 설치 로그 생략)
```

```
In
from langchain_chroma import Chroma
```

In

```
!pip install --upgrade opentelemetry-api
!pip install --upgrade opentelemetry-sdk
```

실행 결과

(패키지 설치 로그 생략)

제일 처음에 사용했던, PDF를 다시 사용하도록 합니다!

In

```
# 위에서 사용했던 코드입니다
loader = PyPDFLoader("/content/Demian.pdf")
pages = loader.load_and_split()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0, length_function = tiktoken_len)
docs = text_splitter.split_documents(pages)
```

In

```
#!pip show chromadb
```

Chroma에 임베딩 시킵니다

In

```
db = Chroma.from_documents(docs, embedding_model)
```

이제 쿼리를 날려보겠습니다

In

```
query = "how Demian look like?"
docs = db.similarity_search(query)
```

In

```
print(docs[0].page_content)
```

실행 결과

```
DEMIAN
with a feeling of nausea, I noticed Demian's expression.
He had not thrust himself to the front but stood right
at the back, looking elegant and at ease as usual. His
glance seemed directed at the horse's head, and again it
. showed that deep, quiet, almost fanatical yet passionate
absorption. I could not help staring at him for some
moments and it was then that I felt aware of a very
uncanny sensation in my remote consciousness. I saw
Demian's face and remarked that it was not a boy's face
but a man's and then I saw, or rather became aware, that
it was not really the face of a man either; it had some
thing different about it, almost a feminine element. And
for the time being his face seemed neither masculine
nor childish, neither old nor young but a hundred years
old, almost timeless and bearing the mark of other
periods of history than our own. Animals might look
thus, trees or stars. I did not know then, of course, I
did not feel exactly what I am writing about it now, as
an adult, but it was something of the kind. Perhaps he
was handsome, perhaps I found him attractive, perhaps
he repelled me too, I could not even be sure of that. All
... (출력 일부 생략)
```

Face, features, looks like 등 데미안의 생김새를 담고 있는 페이지가 출력되었습니다
굉장히 빠른 속도로 검색했습니다!

Step5 : Retriever 사용해보기

Retriever는 사용자의 질문을 Embedding 모델을 통해 벡터로 변환한 뒤, VectorStore에 저장된 문서 벡터들과 비교하여 의미적으로 가장 유사한 문서(Chunk)를 찾아 반환하는 역할을 합니다.

즉, Retriever는

RAG 시스템에서 “어떤 정보를 LLM에게 참고 자료로 줄 것인가”를 결정하는 핵심 컴포넌트이며, 검색 결과의 품질이 곧 최종 답변의 품질로 이어집니다.

In

```
!pip install -U langchain langchain-classic
```

실행 결과

(패키지 설치 로그 생략)

In

```
from langchain_classic.chains.retrieval_qa.base import RetrievalQA
```

긴 문서 전체를 한 번에 LLM에 전달하는 대신, Retriever와 LLM을 결합한 RetrievalQA 체인을 사용하여 문서에서 질문과 관련된 부분만 검색하고, 그 결과를 바탕으로 답변을 생성합니다.

이를 통해 길이가 긴 문서에서도 토른 제한을 넘지 않으면서, 근거 기반의 질의응답을 수행할 수 있습니다.

In

```
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

# OpenAI 모델로 변경
# streaming=True와 callbacks 설정을 통해 실시간 출력을 활성화합니다.
llm = ChatOpenAI(
    model="gpt-4o",           # 또는 "gpt-4o-mini"
    temperature=0.0,
    streaming=True,           # 실시간 출력을 켭니다
    callbacks=[StreamingStdOutCallbackHandler()], # 출력을 콘솔에 바로 뿌려줍니다
)
```

체인의 종류와 검색(Retrieval) 방식, 그리고 그에 따른 주요 파라미터를 설정합니다.

이 단계에서는

Retriever가 어떤 전략으로 문서를 검색할지, 그리고 몇 개의 문서를 LLM에게 전달할지를 결정하게 됩니다. 이 선택은 최종 답변의 품질과 직접적으로 연결됩니다.

예를 들어,

MMR(Maximal Marginal Relevance) 방식은 쿼리와 유사도뿐만 아니라 문서 간의 중복을 줄이고 다양성을 확보하는 재정렬(Re-ranking) 전략입니다.

실무 환경에서는 단일 문서에 정보가 물리는 것을 방지하고, LLM이 보다 풍부한 문맥을 참고하도록 하기 위해 MMR 방식이 자주 사용됩니다.

In

```
qa = RetrievalQA.from_chain_type(llm, chain_type="stuff",
                                  retriever=db.as_retriever(
                                      search_type="mmr",
                                      search_kwargs={"k": 3, "fetch_k": 10}),
                                  return_source_documents=True)
```

위 코드에서 짚고 넘어갈 파라미터는 다음과 같습니다

```
chain_type="stuff"
```

검색된 문서(Chunk)를 그대로 하나의 Prompt에 모두 삽입하는 방식입니다.

구조가 단순하고 이해하기 쉬워,

RAG 구조를 처음 학습하거나 프로토타입을 만들 때 적합합니다.

단점으로는 문서 수가 많아질 경우

토큰 사용량이 빠르게 증가할 수 있습니다.

실무에서는 초기 검증 단계에서는 stuff를,

문서 수가 많아지면 map_reduce나 refine 방식으로 확장합니다.

```
retriever
```

VectorStore에서 어떤 문서를 검색할지 결정하는 검색 모듈입니다.

검색 전략과 파라미터 설정에 따라 LLM이 참고하는 정보의 범위와 품질이 달라집니다.

```
search_type="mmr"
```

MMR(Maximal Marginal Relevance) 검색 방식을 사용합니다. 쿼리와 유사도뿐만 아니라, 문서 간 중복을 줄여 다양한 문맥을 확보하는 Re-ranking 전략입니다. 실무 환경에서 단일 문서 편향을 줄이기 위해 자주 사용됩니다

```
search_kwargs={"k": 3, "fetch_k": 10}
```

```
- fetch_k
```

VectorStore에서 우선적으로 가져올 후보 문서 개수입니다. Re-ranking 이전 단계에서 사용됩니다.

```
- k
```

최종적으로 LLM에게 전달할 문서(Chunk)의 개수입니다.

일반적으로 $fetch_k > k$ 로 설정하여 후보 풀을 넉넉히 확보한 뒤, 품질 좋은 문서만 선별하는 방식을 사용합니다.

```
return_source_documents=True
```

답변 생성에 사용된 원문 문서(Chunk)를 함께 반환합니다. 이를 통해 답변의 출처를 사용자에게 표시하거나 검색 품질을 디버깅하고 RAG 성능을 평가할 수 있습니다. 실무 서비스에서는 거의 필수적으로 사용하는 옵션입니다.

In

```
query = "how demian looks like"
result = qa(query)
```

실행 결과

```
/tmp/ipykernel_12293/3336337621.py:2: LangChainDeprecationWarning: The method `Chain.__call__` was deprecated in langchain-classic 0.1.0 and will be removed in 1.0. Use `invoke` instead.
  result = qa(query)
```

실행 결과

Demian is described as having a face that is neither distinctly masculine nor childish, neither old nor young, but rather timeless and bearing the mark of other periods of history. His face has an almost feminine element and is different from the rest of the people around him. It is suggested that his appearance is unique, almost like an animal, a spirit, or an image, making him unimaginably different from others.

마크다운 형식으로 출력해봅니다

In

```
from IPython.display import Markdown, display
display(Markdown(result["result"]))
```

실행 결과

```
<IPython.core.display.Markdown object>
```

RAG를 사용하지 않은 llm 호출도 시도해보세요!


```
In
llm2 = ChatOpenAI(
    model="gpt-4o")
request = llm2.invoke("how demian looks like")
display(Markdown(request.content))
```

실행 결과

```
<IPython.core.display.Markdown object>
```

Quiz

결과의 어떤 부분을 관찰하였을 때, RAG 시스템의 결과를 신뢰할 수 있겠다 생각하셨나요?

Answer

원문에서 답변의 출처를 확인할 수 있었습니다.

6. 완성 예제

앞에서 진행한 내용으로, Demian을 다시 한번 읽어봅시다!

완성하여 제출해주세요~

필요한 라이브러리를 모두 다운받습니다

```
In
# —— § 6 완성 예제 · 필요한 라이브러리 ——
# 이 섹션은 앞의 Step1~5 셀을 실행하지 않아도 단독으로 돌아가도록 자족적으로 작성했다.
%pip install -qU langchain langchain-core langchain-community langchain-classic \
    langchain-openai langchain-text-splitters langchain-chroma \
    pypdf tiktoken chromadb rank_bm25
```

Text splitter 사용을 위한 준비입니다

In

```
# — 준비 : API 키 · 실험설정(CFG) · 토큰 카운터 · 모델 핸들 —————
import os, re, textwrap, warnings
from pathlib import Path
import numpy as np
warnings.filterwarnings("ignore")

if not os.environ.get("OPENAI_API_KEY"):
    try:
        from google.colab import userdata
        os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")
    except Exception:
        pass
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 먼저 등록하세요."

# 하이퍼파라미터는 이 한 곳에서만 바꾼다.
# RAG는 통제 변수가 많아(chunk_size, overlap, k, fetch_k, λ, 임베딩, 프롬프트)
# 흩어놓으면 성능 변화의 원인을 사후에 귀속시킬 수 없다.
CFG = {
    "PDF_PATH": "/content/Demian.pdf",
    "CHUNK_SIZE": 400, # tokens ← 5 Step2 진단 결과로 500에서 하향 (근거는 해당 셀)
    "CHUNK_OVERLAP": 80, # ≈20%
    "EMBED_MODEL": "text-embedding-3-small",
    "GEN_MODEL": "gpt-4o",
    "TEMPERATURE": 0.0, # 근거 기반 QA의 목표는 창의성이 아니라 재현성
    "K": 4, # LLM에 최종 투입할 chunk 수
    "FETCH_K": 20, # MMR 재순위 이전 후보 풀
    "LAMBDA_MULT": 0.5, # 1.0=유사도만 / 0.0=다양성만
    "BM25_WEIGHT": 0.4, # hybrid 가중치 (BM25 : dense = 0.4 : 0.6)
    "PERSIST_DIR": "/content/chroma_demian",
    "COLLECTION": "demian_v3",
}

import tiktoken
_enc = tiktoken.get_encoding("cl100k_base")
def tiktoken_len(text: str) -> int:
    """문자 수가 아니라 '모델이 실제로 소비하는 토큰 수'로 길이를 잰다."""
    return len(_enc.encode(text))

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
llm = ChatOpenAI(model=CFG["GEN_MODEL"], temperature=CFG["TEMPERATURE"])
embedding_model = OpenAIEmbeddings(model=CFG["EMBED_MODEL"])

print(f"준비 완료 · 생성={CFG['GEN_MODEL']} / 임베딩={CFG['EMBED_MODEL']}")
print(f"토큰 카운터 검증 : tiktoken_len('Hello, Demian!') = {tiktoken_len('Hello, Demian!')}")
```

Step 1 Document loader

```
In
# — Step 1 · Document Loader + OCR 잡음 정제 —————
# load_and_split() 대신 load() 를 쓴다. 기본 splitter가 먼저 끼어들면
# 아래 정제를 적용할 기회를 잃기 때문이다. 분할은 Step 2에서 명시적으로 한다.
from langchain_community.document_loaders import PyPDFLoader

pdf_path = next((p for p in [Path(CFG["PDF_PATH"]), Path("Demian.pdf"),
                             Path("/content/drive/MyDrive/Demian.pdf")] if p.exists()), None)
assert pdf_path, "Demian.pdf 를 Colab(또는 Drive)에 업로드하세요."

pages_raw = PyPDFLoader(str(pdf_path)).load()
print(f"로드 : {len(pages_raw)} pages | producer={pages_raw[0].metadata.get('producer')}")
print(f"      creator={pages_raw[0].metadata.get('creator')} ← ScanFix = 스캔 OCR 산출물\n")

# — 이 PDF에서 실제로 관측되는 OCR 잡음 3종 —————
# (1) 줄바꿈 분철 : invent<soft-hyphen>\ning → 서로 다른 토큰열이 되어 검색 miss
# (2) 러닝헤더/워터마크 : DEMIAN, Downloaded from ... 가 전 페이지 반복
# → 모든 chunk가 서로 비슷해지는 임베딩 anisotropy를 키운다
# (3) 불규칙 공백·빈 페이지 : chunk 예산 낭비
DEHYPH = re.compile(r"([A-Za-z])[-\u00ad\u2010]\s*\n\s*([a-z])")
RUNNING_HEADER = re.compile(
    r"^\s*(DEMIAN|TWO WOR\?.?LDS|CAIN|THE THIEF|BEATRICE|EVA|"
    r"Downloaded from https?://\S+|[·\-\u2010]\s*\d{1,3})\s*$", re.M)

def clean(text: str) -> str:
    # 순서 주의 : 헤더를 먼저 지워야 헤더에 가려져 있던 분철이 드러난다.
    text = RUNNING_HEADER.sub("", text)
    text = DEHYPH.sub(r"\1\2", text)
    text = text.replace("\u00ad", "") # 잔여 soft hyphen
    text = re.sub(r"[\t]+\s+", " ", text)
    text = re.sub(r"\n{3,}", "\n\n", text) # 문단 경계 \n\n 은 splitter가 쓰므로 보존
    return text.strip()

# — 정제 전/후 계측 : "고쳤다"가 아니라 "얼마나 고쳤다"를 남긴다 —————
_all_raw = "\n".join(p.page_content for p in pages_raw)
n_hyph = len(DEHYPH.findall(_all_raw))
n_soft = sum(1 for m in DEHYPH.finditer(_all_raw) if "\u00ad" in m.group(0))
n_wm = _all_raw.count("holybooks")

before = sum(len(p.page_content) for p in pages_raw)
for p in pages_raw:
    p.page_content = clean(p.page_content)
pages = [p for p in pages_raw if tiktoken_len(p.page_content) > 30] # 표지·백지·판권지 제거
after = sum(len(p.page_content) for p in pages)

_all_clean = "\n".join(p.page_content for p in pages)
print(f"분철 복원 : {n_hyph}건 (soft hyphen {n_soft} / ASCII {n_hyph-n_soft}) "
      f"→ 잔존 {len(DEHYPH.findall(_all_clean))}건")
print(f"워터마크 : {n_wm}건 → 잔존 {_all_clean.count('holybooks')}건")
print(f"러닝헤더 : 잔존 {len(re.findall(r'^DEMIAN\s$', _all_clean, re.M))}건")
print(f"분량 : {before:,}자 → {after:,}자 ({100*(before-after)/before:.1f}% 제거)")
print(f"유효 페이지 : {len(pages)} / {len(pages_raw)}")
print(f"- " * 78)
print(pages[8].page_content[:300])
```

| Step 2 Text splitters

In

```
# — Step 2 · Text Splitter —
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

# □ 진단 먼저 : "chunk_size=500"이 이 문서에서 실제로 작동하는가?
page_tokens = [tiktoken_len(p.page_content) for p in pages]
print(f"페이지 토큰 길이 : median {int(np.median(page_tokens))} / "
      f"p95 {int(np.percentile(page_tokens,95))} / max {max(page_tokens)}")
print(f"500토큰을 넘는 페이지 : {sum(t>=500 for t in page_tokens)}개")
print(f"→ 페이지 단위로 split_documents(chunk_size=500) 하면 어떤 페이지도 분할되지 않는다.\n"
      f"  즉 'chunk = page'이고 chunk_overlap=80은 단 한 번도 적용되지 않는다(무효 파라미터).")
_cut = sum(1 for p in pages if not p.page_content.rstrip().endswith(('.', '!', '?', '"')))
print(f"마침표로 끝나지 않는 페이지 : {_cut}/{len(pages)} = {100*_cut/len(pages):.0f}% "
      f"← 문장이 페이지 경계에서 잘려 있다는 뜻\n")

# □ 대응 : 페이지를 한 번 이어붙인 뒤 토큰 기준으로 다시 자른다.
#   페이지 경계는 '종이의 사정'이지 '서사의 경계'가 아니다. 다만 인용 가능성을
#   잃지 않도록, 자른 뒤 문자 오프셋으로 원래 페이지 번호를 되돌려 붙인다.
book, spans, cur = [], [], 0
for p in pages:
    t = p.page_content
    book.append(t)
    spans.append((cur, cur + len(t), p.metadata.get("page")))
    cur += len(t) + 1 # 사이에 넣을 "\n" 1칸
book = "\n".join(book)
book = re.sub(r"([a-z,])\n([a-z])", r"\1 \2", book) # 경계에서 끊긴 문장 잇기(길이 보존)

def _page_of(pos: int):
    for s, e, pg in spans:
        if s <= pos < e:
            return pg
    return spans[-1][2]

splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ". ", " ", ""], # 거친 경계 → 고운 경계
    chunk_size=CFG["CHUNK_SIZE"],
    chunk_overlap=CFG["CHUNK_OVERLAP"],
    length_function=tiktoken_len, # ← 문자 수가 아닌 토큰 수
)
_chunks = splitter.split_text(book)

docs, _pos = [], 0
for i, ch in enumerate(_chunks):
    st = book.find(ch[:80], _pos)
    if st == -1:
        st = max(book.find(ch[:80]), 0)
    p0, p1 = _page_of(st), _page_of(max(st, st + len(ch) - 1))
    docs.append(Document(page_content=ch, metadata={
        "chunk_id": i, "page": p0,
        "page_span": str(p0) if p0 == p1 else f"{p0}-{p1}",
        "source": pdf_path.name}))
    _pos = st + 1

lens = [tiktoken_len(d.page_content) for d in docs]
print(f"chunk 개수 : {len(docs)}")
print(f"토큰 분포 : min {min(lens)} / median {int(np.median(lens))} / "
      f"p95 {int(np.percentile(lens,95))} / max {max(lens)}")
print(f"size 초과 : {sum(l > CFG['CHUNK_SIZE'] for l in lens)}개 ← 0이어야 정상")
print(f"페이지 경계를 넘는 chunk : "
      f" {sum('-' in d.metadata['page_span'] for d in docs)}개 "
      f" ← 페이지 단위 분할이었다면 0개였을 문맥")
print("-" * 78)
print(docs[53].page_content[:300])
```

Step 3 Vector Empeddings

```
In
# — Step 3 · Embeddings (인덱싱 전에 값싼 sanity check 먼저) —————
# 전체 인덱싱은 API 비용과 시간이 든다. "이 임베딩이 우리 도메인에서
# 의미 구분을 하는가"를 3문장으로 먼저 확인하고 넘어간다.
from numpy import dot
from numpy.linalg import norm
cos_sim = lambda A, B: dot(A, B) / (norm(A) * norm(B))

probe_texts = [
    docs[53].page_content,                # 실제 본문 chunk
    "Demian's face was neither a boy's nor a man's, almost timeless.", # 외모 묘사 → 가까워야
    "The recipe requires two cups of flour and a pinch of salt.",      # 무관 → 멀어야
]
E = embedding_model.embed_documents(probe_texts)
q = embedding_model.embed_query("What did Demian's face look like?")

print(f"임베딩 차원          : {len(E[0])}")
print(f"query ↔ 본문 chunk    : {cos_sim(E[0], q):.4f}")
print(f"query ↔ 외모 묘사 문장  : {cos_sim(E[1], q):.4f} ← 가장 높아야 정상")
print(f"query ↔ 무관한 레시피 문장 : {cos_sim(E[2], q):.4f} ← 가장 낮아야 정상")
```

```
In
# — Step 3-b · VectorStore 구축 (Chroma, 재실행 안전) —————
# from_documents() 는 embed_documents → 벡터 저장 → HNSW 인덱스 구성을 한 번에 한다.
# persist_directory 를 주면 세션이 끊겨도 같은 인덱스 위에서
# retriever·프롬프트만 바뀌가며 실험할 수 있다.
from langchain_chroma import Chroma
import chromadb

try:    # 같은 셀을 두 번 실행하면 동일 문서가 중복 적재된다 → 먼저 비운다
    chromadb.PersistentClient(path=CFG["PERSIST_DIR"]).delete_collection(CFG["COLLECTION"])
    print(f"기존 컬렉션 '{CFG['COLLECTION']}' 삭제")
except Exception:
    pass

db = Chroma.from_documents(
    documents=docs,
    embedding=embedding_model,
    collection_name=CFG["COLLECTION"],
    persist_directory=CFG["PERSIST_DIR"],
)
try:
    n_vec = db._collection.count()
except Exception:
    n_vec = len(docs)
print(f"인덱싱 완료 : {n_vec:,} vectors (chunk {len(docs)}개와 일치해야 정상)")
```

Step 4 Retrievers

```
In
# — Step 4 · Retrievers : similarity / MMR / BM25 / Hybrid —————
# Retriever는 "LLM에게 무엇을 보여줄 것인가"를 결정한다. 검색이 틀리면 생성은 반드시 틀린다.
from langchain_community.retrievers import BM25Retriever
try:
    from langchain_classic.retrievers import EnsembleRetriever
except ImportError:
    from langchain.retrievers import EnsembleRetriever

sim_retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": CFG["K"]})

mmr_retriever = db.as_retriever(
    search_type="mmr",
    search_kwargs={"k": CFG["K"], "fetch_k": CFG["FETCH_K"],
                  "lambda_mult": CFG["LAMBDA_MULT"]}) # 유사도 - λ·기존 선택과의 중복 으로 재순위

bm25_retriever = BM25Retriever.from_documents(docs) # 어휘 정확 매칭(sparse)
bm25_retriever.k = CFG["K"]

hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, mmr_retriever],
    weights=[CFG["BM25_WEIGHT"], 1 - CFG["BM25_WEIGHT"]]) # RRF로 두 순위를 융합

def show(name, retriever, query, n=4):
    hits = retriever.invoke(query)[:n]
    print(f"[{name:9s}] 페이지 {[h.metadata.get('page_span') for h in hits]}")
    for h in hits:
        head = h.page_content.replace("\n", " ")[0:78]
        print(f"    ↳ p.{str(h.metadata.get('page_span')):>7} #{h.metadata['chunk_id']:>4} | {head}...")

for q in ["How does Demian look like?", "Who is Abraxas?"]:
    print("=" * 78); print("QUERY:", q); print("=" * 78)
    for nm, r in [("similarity", sim_retriever), ("mmr", mmr_retriever),
                  ("bm25", bm25_retriever), ("hybrid", hybrid_retriever)]:
        show(nm, r, q)
    print()
```

In

```
# — Step 4-b · 네 retriever를 같은 자로 재기 (Recall@k · MRR) —
# "hybrid가 좋다"는 인상이 아니라 수치로 확인한다. 정답 chunk는 원문의 고유 문자열로
# 지정한다(문자열 매칭이므로 판정에 LLM이 개입하지 않아 재현 가능하다).
GOLD = {
    "How does Demian look like?": "not a boy's face", # 외모 묘사
    "Who is Abraxas?": "Abraxas", # 고유명사·저빈도
    "What did Franz Kromer demand from Sinclair?": "two marks", # 사실 회수
    "What are the two worlds Sinclair describes?": "two worlds", # 상징
}

def evaluate_retrievers(retrievers, gold=GOLD, k=CFG["K"]):
    rows = []
    for name, r in retrievers:
        hit, rr = 0, []
        for q, key in gold.items():
            got = r.invoke(q)[:k]
            pos = next((i + 1 for i, d in enumerate(got)
                        if key.lower() in d.page_content.lower()), None)
            hit += pos is not None
            rr.append(1 / pos if pos else 0.0)
        rows.append((name, hit / len(gold), float(np.mean(rr))))
    return rows

print(f'{"retriever":<12}{"Recall@'+str(CFG['K'])+'>10}{"MRR":>8}')
print("-" * 30)
for name, rec, mrr in evaluate_retrievers(
    [("similarity", sim_retriever), ("mmr", mmr_retriever),
     ("bm25", bm25_retriever), ("hybrid", hybrid_retriever)]):
    print(f'{"name":<12}{"rec:>10.2f"}{"mrr:>8.3f}")

# 해석 메모(오프라인 사전 측정 결과) —
# BM25 단독은 "Who is Abraxas?"에서 정답을 1위로 올리지만(저빈도 고유명사),
# "How does Demian look like?"에서는 정답이 38위로 밀린다(질문이 불용어 중심).
# dense 단독은 그 반대다. 두 실패 모드가 상보적이기 때문에 앙상블이 의미를 갖는다.
# → 가중치 0.4/0.6은 이 표를 키운 뒤 grid search로 확정해야 할 값이지, 아직은 가설이다.
```


Step 5 Question Answering

In

```
# — Step 5 · Question Answering (Grounding 프롬프트 + LCEL) —
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# ① Grounding — RAG의 실패는 대개 "검색은 됐는데 모델이 자기 사전지식으로 덧칠"할 때 생긴다.
# 『데미안』은 학습 코퍼스에 거의 확실히 들어 있는 정전(canon)이라 이 위험이 특히 크다.
# ② 출처 표기 — §Quiz의 답("원문에서 출처를 확인할 수 있었다")을 시스템 차원에서 보장한다.
SYSTEM = """당신은 헤르만 헤세의 소설 『데미안』 원문만을 근거로 답하는 문학 연구 조교입니다.

반드시 지킬 규칙:
1. 아래 <context> 안의 내용만 근거로 삼습니다. 이미 알고 있는 『데미안』 지식으로
   보충하거나 추론으로 메우지 마십시오.
2. context에 근거가 없으면 "제공된 원문에서는 확인할 수 없습니다"라고 답하고,
   무엇이 부족한지 한 문장으로 덧붙입니다. 추측하지 마십시오.
3. 각 주장 끝에 근거를 [p.{{페이지}}] #{{chunk_id}} 형식으로 표기합니다.
4. 한국어로, 3~6문장으로 답합니다.
5. 원문 인용은 한 문장 이내로 짧게 하고 즉시 출처를 답니다."""

USER = """<context>
{context}
</context>

질문: {question}"""

prompt = ChatPromptTemplate.from_messages([("system", SYSTEM), ("human", USER)])

def format_docs(ds) -> str:
    """검색된 chunk를 출처 헤더와 함께 직렬화한다.
    모델이 인용할 수 있도록 메타데이터를 본문과 같은 평문 안에 넣는 것이 핵심."""
    return "\n\n---\n\n".join(
        f"[p.{d.metadata.get('page_span')}] #{d.metadata.get('chunk_id')}] \n{d.page_content}"
        for d in ds)

rag_chain = (
    RunnableParallel(context=hybrid_retriever | format_docs,
                     question=RunnablePassthrough())
    | prompt | llm | StrOutputParser()
)

def ask(question: str, retriever=hybrid_retriever, verbose: bool = True):
    """return_source_documents=True 와 같은 목적 — 답변과 근거를 함께 돌려준다."""
    sources = retriever.invoke(question)
    answer = (prompt | llm | StrOutputParser()).invoke(
        {"context": format_docs(sources), "question": question})
    if verbose:
        print("Q:", question); print("-" * 78)
        print(textwrap.fill(answer, 78)); print("-" * 78)
        print("근거 chunk:")
        for s in sources:
            head = s.page_content.replace("\n", " ").[:66]
            print(f" • p.{str(s.metadata.get('page_span'))}>7} #{s.metadata['chunk_id']>4} | {head}...")
        print()
    return {"answer": answer, "sources": sources}

_ = ask("데미안의 외모는 어떻게 묘사되는가?")
```

```

In
# — Step 5-b · 검증 : ① 대조군 ② 환각 저항성 ③ 배치 질의 —————
# RAG를 붙였다는 사실만으로는 아무것도 증명되지 않는다.

# ① 대조군 — 같은 질문을 RAG 없이. 원문 근거를 지목하는가 vs 일반론으로 흐르는가.
llm_plain = ChatOpenAI(model=CFG["GEN_MODEL"], temperature=0.0)
print("┌" + "─" * 76 + "┐")
print("│ [No-RAG] 모델 내부 지식만 사용 " + " " * 45 + " │")
print("└" + "─" * 76 + "┘")
print(textwrap.fill(llm_plain.invoke("데미안의 외모는 어떻게 묘사되는가?").content, 78), "\n")

# ② 환각 저항성 — 원문에 없는 사실을 물었을 때 거절하는가.
# 거절하지 못하면 그 시스템의 '맞은 답변'조차 우연인지 실력인지 구분할 수 없다.
print("┌" + "─" * 76 + "┐")
print("│ [Negative probe] 원문에 없는 정보 — 거절해야 정상 " + " " * 27 + " │")
print("└" + "─" * 76 + "┘")
for p in ["데미안이 졸업한 대학의 정확한 이름과 졸업 연도는?",
          "싱클레어의 여동생이 결혼한 상대의 직업은 무엇인가?"]:
    _ = ask(p)

# ③ 배치 질의 — 난이도가 다른 5문항으로 파이프라인 전체를 한 번에 통과시킨다.
QUESTIONS = [
    "Franz Kromer는 싱클레어에게 무엇을 요구했는가?",          # 사실 회수(단일 chunk)
    "데미안이 카인(Cain)의 이야기를 새롭게 해석한 방식은?",     # 관계 추론(다중 chunk)
    "Abraxas는 이 소설에서 어떤 의미를 지니는가?",              # 상징 해석
    "싱클레어가 말하는 '두 세계(two worlds)'란 무엇인가?",      # 한국어 질의 · 영어 원문
    "이 소설에서 데미안이 사망한 정확한 날짜는 언제인가?",      # 부재 정보 → 거절
]
results = []
for i, q in enumerate(QUESTIONS, 1):
    print(f"\n┌─*78┐\n│{i}/{len(QUESTIONS)}│")
    results.append(ask(q))

```

7. (확장) 정량 평가 — RAGAS

레퍼런스 노트북 [Rag_evaluation](#) 의 프레임워크를 이 파이프라인에 적용한다.

RAG 평가는 검색과 생성을 분리해서 봐야 원인을 귀속시킬 수 있다.

지표	무엇을 재는가	낮을 때 손덜 곳
<code>context_precision</code>	회수된 chunk 중 실제로 쓸모 있는 비율	<code>k</code> 축소, MMR <code>λ</code> , reranker
<code>context_recall</code>	정답에 필요한 정보를 빠짐없이 회수했는가	<code>chunk_size / overlap</code> , <code>fetch_k</code> , hybrid 가중치
<code>faithfulness</code>	답변이 context에서만 나왔는가 (= 환각의 역수)	grounding 프롬프트, <code>temperature</code>
<code>answer_relevancy</code>	답변이 질문에 실제로 답했는가	프롬프트, 생성 모델

진단 규칙 — **recall**이 낮으면 **chunking/검색 문제**, **faithfulness**가 낮으면 **프롬프트/생성 문제**.

이 둘을 뭉뚱그리면 엉뚱한 곳을 튜닝하게 된다.

```

In
%pip install -qU ragas datasets
import nest_asyncio; nest_asyncio.apply()      # Colab 이벤트 루프 충돌 방지

from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall, context_precision
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

eval_questions = [
    "Franz Kromer는 싱글레어에게 무엇을 요구했는가?",
    "싱글레어가 말하는 '두 세계(two worlds)'란 무엇인가?",
    "이 소설에서 데미안이 사망한 정확한 날짜는 언제인가?",
]
eval_ground_truths = [
    "크로머는 싱글레어가 지어낸 사과 도둑질 이야기를 빌미로 그를 협박하며 돈(2마르크)을 요구했고, "
    "싱글레어는 그 요구에 시달리며 죄책감과 공포에 빠져든다.",
    "밝고 질서 있는 부모의 세계(경건함·규율·안전)와, 그 바깥의 어둡고 금지된 세계"
    "(하인·범죄·성·공포)라는 두 개의 대립된 영역을 뜻한다.",
    "원문에는 데미안의 사망 날짜가 제시되지 않는다.",
]

records = {"user_input": [], "response": [], "retrieved_contexts": [], "reference": []}
for q, gt in zip(eval_questions, eval_ground_truths):
    r = ask(q, verbose=False)
    records["user_input"].append(q)
    records["response"].append(r["answer"])
    records["retrieved_contexts"].append([d.page_content for d in r["sources"]])
    records["reference"].append(gt)

# 판사(judge) 모델은 생성 모델과 분리 — 자기 답을 자기가 채점하는 편향을 줄인다.
evaluator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-mini", temperature=0))
evaluator_emb = LangchainEmbeddingsWrapper(OpenAIEmbeddings(model=CFG["EMBED_MODEL"]))

result = evaluate(
    dataset=Dataset.from_dict(records),
    metrics=[context_precision, context_recall, faithfulness, answer_relevancy],
    llm=evaluator_llm, embeddings=evaluator_emb,
)
print(result)
result.to_pandas()

```

8. 정리 · 한계 · 다음 단계

파이프라인 (첨부 다이어그램의 A~D / 1~5에 대응)

Demian.pdf (스캔 OCR · 182p)

- | PyPDFLoader.load() ← 페이지 = Document
- | clean() 분철 373건 복원 · 러닝헤더/워터마크 제거 · 공백 정규화 [A]
- ▼ 유효 180p
- 페이지 병합 → RecursiveCharacterTextSplitter(400 tok / overlap 80, length_function=tiktoken_len) → 오프셋으로 page 복원 [B]
- ▼ 191 chunks
- OpenAIEmbeddings(text-embedding-3-small, 1536d) [C]
- ▼
- Chroma (persist, HNSW) [D]
- ▼
- Hybrid Retriever = BM25(0.4) ⊕ MMR(0.6, k=4, fetch_k=20, λ=0.5) [1→3]
- ▼
- Grounding Prompt (context-only · 근거 없으면 거절 · [p.#] 인용) [4]
- ▼



이번에 새로 확인한 것 (측정 기반)

- chunk_size=500** 은 이 문서에서 무효 파라미터였다. 페이지 최대 길이가 약 370토큰이라 페이지 단위 분할에서는 어떤 페이지도 잘리지 않는다 → **chunk = page** , **overlap=80** 은 단 한 번도 적용되지 않는다. 파라미터를 적어두는 것과 파라미터가 작동하는 것은 다르다.
- 페이지의 78%가 문장 중간에서 끝난다.** 페이지 경계는 종이의 사정이지 서사의 경계가 아니다. 병합 후 재분할로 이 손실을 없앴고, 오프셋 역매핑으로 인용 가능성은 유지했다.
(예: 「쓰러진 말 → 데미안의 얼굴」 묘사가 이제 한 chunk 안에 들어온다.)
- 정제 규칙의 오탐 위험은 낮다.** 분철 후보 373건 중 369건이 soft hyphen(U+00AD)으로, 조판 하이픈이 아니라 OCR 산출물이었다. 규칙 적용 근거를 수치로 확인했다.
- BM25와 dense의 실패 모드는 상보적이다.** 사전 측정에서 BM25는 "Who is Abraxas?"를 1위로 올렸지만 "How does Demian look like?"에서는 정답이 38위였다(질문이 불용어 중심). 양상블의 근거는 "hybrid가 좋으니까"가 아니라 이 상보성이다.

한계

- 평가셋이 3~4문항이라 신뢰구간이 없다. 최소 30~50문항, 난이도 계층별 층화가 필요하다.
- 가중치 **[0.4, 0.6]** , $\lambda=0.5$, $k=4$ 는 아직 **가설**이다. 평가셋을 키운 뒤 one-factor-at-a-time grid search로 확정해야 한다. (한 번에 여러 파라미터를 바꾸면 원인 귀속이 불가능해진다.)
- chunk 경계가 여전히 **장(chapter)·장면** 단위와 어긋난다 → semantic chunking 또는 parent-child 인덱싱이 다음 후보다.
- 비용/지연 관측이 없다. 답변 1건당 토큰·지연·비용을 로깅해야 품질-비용 곡선을 그릴 수 있다.

다음 단계

방향	기법	기대 효과
인덱싱	Semantic chunking, Parent-Document Retriever	chunk 경계 손실 제거
쿼리	Multi-Query, HyDE, Query Decomposition	모호한 질의 recall ↑
증강	Cross-encoder Reranker (bge-reranker 등)	precision ↑ , k 축소 가능
구조	Self-RAG / CRAG (검색 필요 여부를 모델이 판단)	불필요 검색 제거, 환각 ↓
평가	50문항 + LLM-as-judge 이중 채점	튜닝 결과의 유의성 확보

완성 제출 · 추병곤 · Demian RAG Pipeline (LangChain · Chroma · Hybrid Retrieval · RAGAS)